

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5

Total Marks: 50

Annexure A — Code of Conduct

I **_PERAM VIVEKANANDA REDDY(192372330)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Vivek*

Annexure A — Design Task (Question Paper)

1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

Rubrics for Calculation

Criteria	Weightage	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	10%	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	20%	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	15%	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	15%	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	10%	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed

Criteria	Weightage	Excellent (5)	Good (4)	Average (3)	Poor (2)
Data Flow & Processing Pipeline	10%	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	10%	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	10%	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

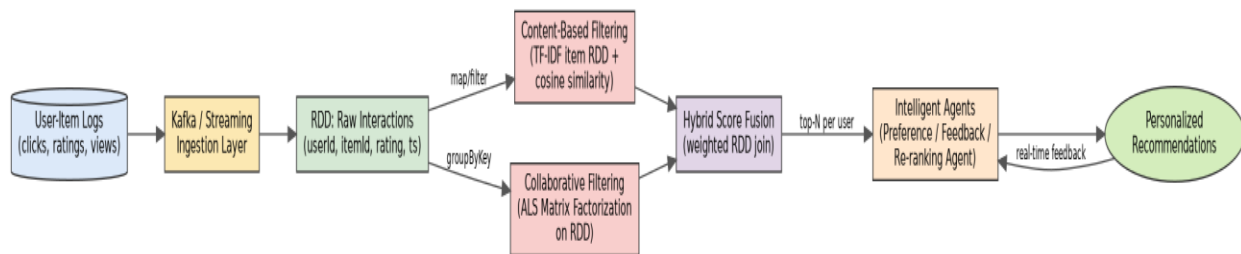
ANSWERS

1. Hybrid AI Recommendation Engine Design

Problem Understanding

An e-commerce or media platform needs to recommend items to users at scale, using both (i) content-based filtering — matching item attributes to a user's known preferences — and (ii) collaborative filtering — leveraging patterns across similar users' behaviour. The challenge is combining both signals into one hybrid score, over potentially billions of user-item interaction events, while adapting in near real time as users click, rate, or ignore recommendations.

System Architecture



Raw interaction events (clicks, ratings, purchases, dwell-time) are streamed through Kafka into Spark, forming an RDD of (userId, itemId, rating, timestamp) records. Two parallel pipelines then run over this RDD: a content-based branch that builds a TF-IDF vector per item (from item metadata/descriptions) and computes cosine similarity to a user's historical item profile, and a collaborative-filtering branch that uses Alternating Least Squares (ALS) matrix factorization on the user-item rating matrix (natively RDD-based in Spark MLlib) to learn latent user/item factors. A hybrid fusion stage joins both RDDs and computes a weighted score per candidate item, which is then passed to the intelligent agent layer for final re-ranking and delivery.

PySpark / RDD Operations

```
from pyspark import SparkContext
from pyspark.mllib.recommendation import ALS, Rating

sc = SparkContext(appName='HybridRecommender')
raw_rdd = sc.textFile('s3://logs/interactions/*.csv')

# Parse into (userId, itemId, rating) RDD
ratings_rdd = raw_rdd.map(lambda line: line.split(',')) \
    .map(lambda p: Rating(int(p[0]), int(p[1]), float(p[2])))

# --- Collaborative filtering branch: distributed ALS ---
cf_model = ALS.train(ratings_rdd, rank=20, iterations=10, lambda_=0.01)

# --- Content-based branch: item profile similarity ---
item_profiles_rdd = sc.textFile('s3://catalog/items/*.json') \
    .map(build_tfidf_vector) # user-defined map function
user_history_rdd = ratings_rdd.map(lambda r: (r.user, r.product)) \
    .groupByKey()
cbf_scores_rdd = user_history_rdd.join(item_profiles_rdd) \
    .mapValues(cosine_similarity_score)

# --- Hybrid fusion: weighted RDD join ---
hybrid_rdd = cf_model.predictAll(candidate_pairs_rdd) \
    .map(lambda r: ((r.user, r.product), r.rating)) \
    .join(cbf_scores_rdd) \
    .mapValues(lambda x: 0.6 * x[0] + 0.4 * x[1]) # weighted fusion
```

Key RDD operations used: map (parsing and feature extraction), groupByKey (aggregating a user's interaction history), join (combining CF and CBF scores per user-item pair), and mapValues (applying the fusion formula) — all of which Spark automatically partitions and executes in parallel across the cluster.

Role of Intelligent Agents

- Preference Agent: maintains a lightweight, continuously-updated per-user preference vector, incorporating each new click/rating instantly (rather than waiting for the next full batch ALS retrain), so recommendations reflect the user's most recent session.
- Feedback Agent: monitors implicit signals (skip, dwell-time, purchase) after each recommendation is shown, converting them into reward signals used to adjust the hybrid fusion weight (α between CF and CBF) per user — users with rich rating history lean more on CF, cold-start users lean more on CBF.
- Re-ranking Agent: applies final business-logic constraints (diversity, freshness, inventory availability) to the top-N hybrid-scored candidates before display, and closes the loop by feeding the user's reaction back to the Feedback Agent — this real-time agent loop is what lets the system adapt without waiting for the next full distributed retraining cycle.

Scalability & Innovation

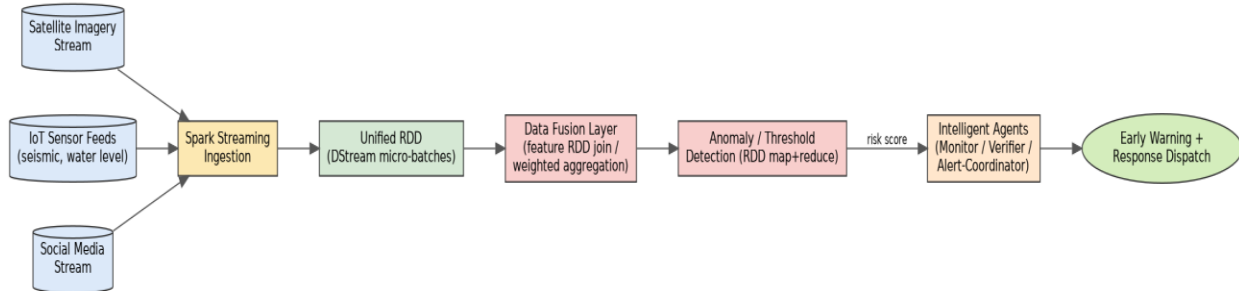
- Scalability: ALS and the TF-IDF similarity computation are both natively data-parallel across RDD partitions keyed by user/item ID, so the system scales horizontally by adding worker nodes; Spark's lineage-based fault tolerance means a failed partition is simply recomputed from its lineage rather than requiring a full restart.
- Innovation: rather than a single static hybrid weight, the fusion coefficient is personalised per user by the Feedback Agent based on their interaction-history depth, effectively giving new users more content-based (cold-start-robust) recommendations and long-tenured users more collaborative (community-pattern-based) ones automatically.

2. Smart Disaster Detection System

Problem Understanding

Natural disasters (floods, earthquakes) must be detected as early as possible by fusing heterogeneous, high-velocity data sources — satellite imagery (large, infrequent, high-latency), sensor telemetry (small, frequent, low-latency), and social media posts (unstructured, bursty, noisy) — into a single, low-latency early-warning decision, at a scale that would overwhelm any single-machine pipeline during an actual crisis surge in data volume.

System Architecture



Each data source streams into a common Spark Streaming ingestion layer, which converts incoming micro-batches into RDDs (technically DStreams, i.e., a continuous sequence of RDDs) on a common time window (e.g., every 30 seconds). A data-fusion stage joins/aggregates these heterogeneous RDDs into a unified per-region feature vector (e.g., region → {seismic reading, water-level trend, social-media disaster-keyword frequency, satellite change-detection score}). An anomaly/threshold-detection stage then flags regions whose fused feature vector crosses a learned or statistical threshold, and the intelligent agent layer verifies and coordinates the resulting alert.

Data Fusion Technique

Because the three sources arrive at very different rates and formats, fusion is done via a common spatial-temporal key (region ID + time-window) rather than trying to align records one-to-one: each source is independently mapped to (region, window) → partial-feature-vector, then a single `reduceByKey` merges all partial vectors for the same key into one combined vector — a classic distributed map-reduce fusion pattern that scales linearly with the number of source records, not the number of sources.

```
# Each source stream -> (region, window) keyed partial feature
sat_features = sat_rdd.map(lambda r: ((r.region, r.window), {'sat_score': r.change_score}))
sensor_features = sensor_rdd.map(lambda r: ((r.region, r.window), {'seismic': r.magnitude}))
social_features = social_rdd.map(lambda r: ((r.region, r.window), {'keyword_freq': r.freq}))

fused_rdd = sat_features.union(sensor_features).union(social_features) \
    .reduceByKey(lambda a, b: {**a, **b}) # merge partial dicts

alerts_rdd = fused_rdd.filter(lambda kv: risk_score(kv[1]) > THRESHOLD)
```

Role of Intelligent Agents

- **Monitor Agent:** continuously watches the stream of fused, thresholded RDD outputs per region and maintains short-term trend state (is the risk score rising or falling), to avoid firing on a single noisy spike.
- **Verifier Agent:** cross-checks a candidate alert against at least two independent sources (e.g., sensor spike + corroborating social-media mentions) before escalating, reducing false positives from any single noisy channel.
- **Alert-Coordinator Agent:** once verified, dispatches region-specific warnings to the relevant emergency-response channels and can dynamically re-prioritise which regions receive additional satellite tasking/compute resources, effectively closing a decentralized feedback loop between detection and response.

Scalability & Innovation

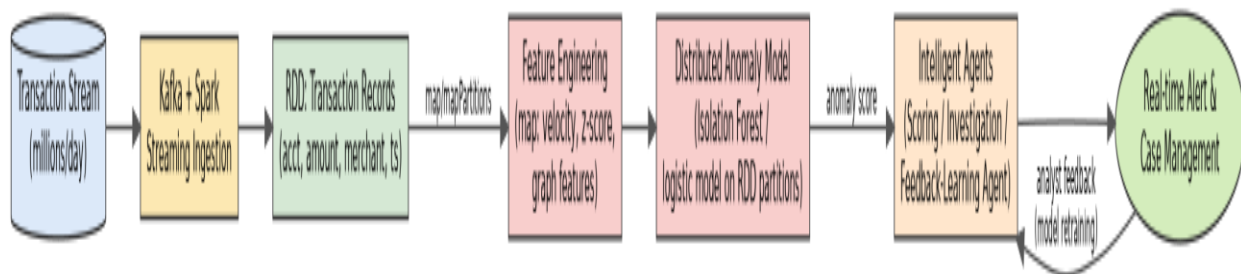
- Scalability: partitioning by region ID means each region's fusion and anomaly detection runs independently and in parallel; Spark Streaming's micro-batch model provides natural fault tolerance (a failed batch is simply reprocessed from the source offset).
- Innovation: using a common (region, time-window) key to fuse fundamentally different data modalities (imagery, sensors, text) avoids the need for expensive record-level multimodal alignment, making the fusion step itself embarrassingly parallel and easy to extend with new data sources.

3. AI-Based Fraud Detection Framework

Problem Understanding

A payments platform must screen potentially millions of transactions per hour for fraud, where fraud patterns are rare (highly imbalanced), evolve over time (adversarial, fraudsters adapt), and detection must happen within milliseconds to seconds to block a transaction before settlement — requiring a distributed, continuously-updating anomaly-detection pipeline rather than a periodic batch model.

System Architecture



Transactions stream through Kafka into Spark Streaming, forming an RDD per micro-batch. A feature-engineering stage computes per-account behavioural features (transaction velocity, deviation from typical spend z-score, merchant-category entropy, and graph-based features such as shared-device/shared-IP links between accounts) using `mapPartitions` for efficiency (so expensive feature computations reuse per-partition state rather than being recomputed per record). A distributed anomaly-scoring stage (e.g., an Isolation Forest or a logistic/gradient-boosted model, trained offline and broadcast to all worker nodes) scores each transaction; intelligent agents then decide whether to alert, hold, or auto-approve.

Distributed Anomaly Detection

```
from pyspark import SparkContext
sc = SparkContext(appName='FraudDetection')

tx_rdd = sc.textFile('kafka://transactions').map(parse_transaction)

# Feature engineering per partition (avoids recomputing shared state per record)
def compute_features(partition):
    account_state = load_account_profiles() # loaded once per partition, not per record
    for tx in partition:
        yield (tx.acct_id, extract_features(tx, account_state))

feature_rdd = tx_rdd.mapPartitions(compute_features)

# Broadcast the trained anomaly model to every worker node (avoids re-shipping per task)
broadcast_model = sc.broadcast(trained_isolation_forest)
scored_rdd = feature_rdd.map(lambda kv: (kv[0], broadcast_model.value.score(kv[1])))

suspicious_rdd = scored_rdd.filter(lambda kv: kv[1] > ANOMALY_THRESHOLD)
```

Broadcasting the trained model (rather than re-loading it per task) and using `mapPartitions` (rather than `map`) for stateful feature extraction are both standard distributed-computing optimisations that meaningfully reduce per-record overhead at this transaction volume.

Role of Intelligent Agents

- **Scoring Agent:** wraps the distributed model's output with account-specific context (e.g., a normally-high-spender's large transaction is less suspicious than the same amount for a typically-low-spender), adjusting the raw anomaly score before it is acted on.

- Investigation Agent: for borderline scores, automatically gathers supporting evidence (recent account activity, device/IP graph neighbours) and routes the case to a human analyst dashboard with a pre-built explanation, rather than a bare number.
- Feedback-Learning Agent: captures analyst decisions (confirmed fraud / false alarm) and feeds them back into a continuously-updated training set, triggering periodic distributed model retraining so detection accuracy improves as fraud patterns evolve — directly addressing the requirement to 'continuously improve detection accuracy.'

Scalability & Innovation

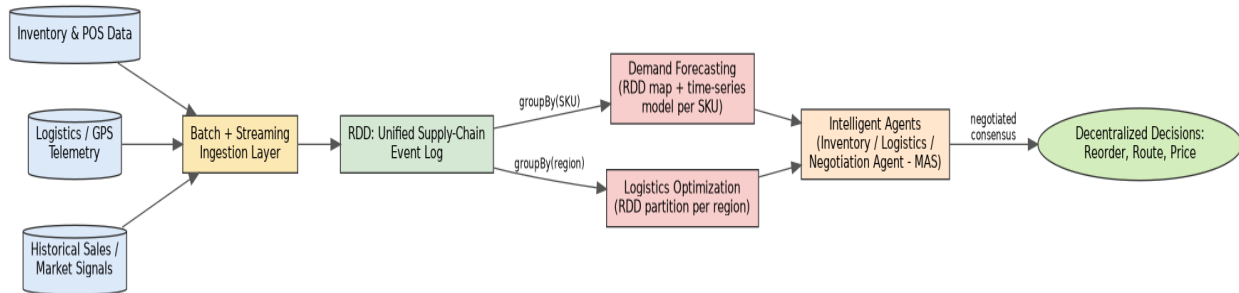
- Scalability: partitioning transactions by account ID keeps each account's state colocated, so mapPartitions-based feature computation avoids cross-partition shuffles for the most latency-sensitive features; the broadcast model avoids repeated network transfer to each of potentially thousands of executor tasks.
- Innovation: closing the loop from analyst feedback directly into scheduled distributed retraining (rather than a manual, infrequent model refresh) lets the system adapt to new fraud patterns within hours rather than months.

4. Autonomous Supply Chain Optimization System

Problem Understanding

A multi-warehouse, multi-region retailer needs to jointly optimise inventory levels, delivery routing, and demand forecasting across thousands of SKUs and regions simultaneously — a combinatorially large decision space where centralised, single-machine optimisation would be both computationally infeasible and too slow to react to daily demand shifts, motivating a distributed, multi-agent decentralised-decision architecture instead.

System Architecture



Inventory/POS data, logistics/GPS telemetry, and historical demand signals are ingested into a unified RDD of supply-chain events. This RDD is grouped by SKU for demand forecasting (each SKU's time series is forecast independently and in parallel) and separately grouped by region for logistics/route optimisation (each region's vehicle-routing problem is solved independently). The resulting per-SKU forecasts and per-region routing plans feed a multi-agent system (MAS) where distinct agent types negotiate final decisions.

RDD Transformations for Large-Scale Processing

```
from pyspark import SparkContext
sc = SparkContext(appName='SupplyChainOptimizer')

events_rdd = sc.textFile('s3://scm/events/*.json').map(parse_event)

# Demand forecasting: one time-series model fit per SKU, run in parallel
sku_series_rdd = events_rdd.map(lambda e: (e.sku_id, e)) \
    .groupByKey() \
    .mapValues(lambda events: fit_forecast_model(list(events)))

# Logistics optimisation: one routing sub-problem solved per region, run in parallel
region_routes_rdd = events_rdd.map(lambda e: (e.region_id, e)) \
    .groupByKey() \
    .mapValues(lambda events: solve_vehicle_routing(list(events)))

# Combine into a single decision RDD keyed by (region, sku)
decisions_rdd = sku_series_rdd.cartesian(region_routes_rdd) \
    .filter(relevant_pairs_only) \
    .map(build_decision_record)
```

`groupByKey` (by SKU and by region) is what makes both the forecasting and routing stages embarrassingly parallel — each key's sub-problem is entirely independent of every other key's, so Spark can schedule thousands of small forecasting/routing jobs across the cluster with no cross-task communication needed until the final decision-fusion step.

Role of Intelligent Agents (Multi-Agent System)

- Inventory Agent (per warehouse): decides local reorder points and quantities from its SKU forecast, balancing holding cost against stockout risk.
- Logistics Agent (per region): owns the vehicle-routing decisions for its region and can request inventory transfers from neighbouring warehouses when local stock is projected to be insufficient.

- **Negotiation Agent:** mediates when Inventory and Logistics agents' local decisions conflict (e.g., two regions both want stock from the same warehouse), running a lightweight auction/negotiation protocol to reach a decentralised consensus without needing a single global optimiser — this is what makes the system's decision-making genuinely decentralised rather than merely distributed computation feeding one central planner.

Scalability & Innovation

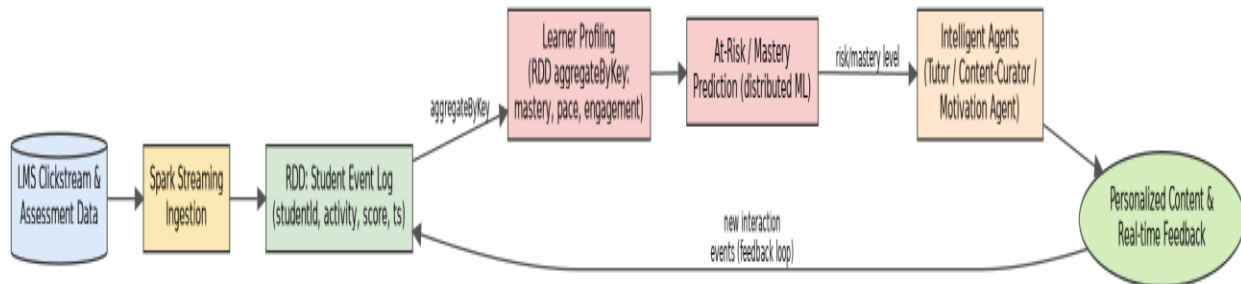
- **Scalability:** because SKU-level forecasting and region-level routing are independent sub-problems, adding more SKUs/regions scales the workload linearly with available cluster capacity, and a failed worker simply causes Spark to recompute the lost partition's SKU/region sub-problems from the RDD lineage.
- **Innovation:** the negotiation-based multi-agent consensus (rather than one centralised linear-programming solver over the entire network) mirrors how real decentralised organisations operate, and scales far better since it avoids ever needing to materialise one enormous joint optimisation problem.

5. Personalized Learning Analytics Platform

Problem Understanding

An online learning platform serving thousands of concurrent learners needs to continuously analyse each student's activity (video views, quiz attempts, time-on-task) to detect mastery level and disengagement risk, and then personalise content delivery and feedback in near real time — a workload that scales with both the number of learners and the granularity of interaction events, again motivating a distributed, streaming-first architecture.

System Architecture



Clickstream and assessment events stream from the LMS into Spark Streaming, forming an RDD of (studentId, activity, score, timestamp) records. A learner-profiling stage aggregates each student's rolling behaviour (mastery per topic, pace relative to peers, engagement trend) using `aggregateByKey`, which is more efficient than `groupByKey` here since it combines values within a partition before any shuffle. A prediction stage flags at-risk students and estimates topic-level mastery; intelligent agents then personalise the next piece of content and feedback shown to each learner, with their subsequent interactions feeding straight back into the same RDD pipeline.

PySpark / RDD Operations

```
from pyspark import SparkContext
sc = SparkContext(appName='LearningAnalytics')

events_rdd = sc.textFile('kafka://lms-events').map(parse_learning_event)

# aggregateByKey: combine per-partition first (zero value, seqFunc, combFunc)
def seq_func(acc, event):
    acc['attempts'] += 1
    acc['correct'] += event.is_correct
    acc['time_spent'] += event.duration
    return acc

def comb_func(acc1, acc2):
    return {k: acc1[k] + acc2[k] for k in acc1}

zero_value = {'attempts': 0, 'correct': 0, 'time_spent': 0}
student_profile_rdd = events_rdd.map(lambda e: (e.student_id, e)) \
    .aggregateByKey(zero_value, seq_func, comb_func)

mastery_rdd = student_profile_rdd.mapValues(estimate_mastery_and_risk)
at_risk_rdd = mastery_rdd.filter(lambda kv: kv[1]['risk_flag'] == True)
```

Role of Intelligent Agents

- **Tutor Agent:** interprets each student's current mastery-per-topic vector and selects the next best learning activity (review vs. advance vs. alternate explanation style), personalising the learning path in real time as new events arrive.

- Content-Curator Agent: matches available content items (videos, practice sets, articles) to the specific skill gaps identified by the Tutor Agent, and can request new content tags/metadata be generated for under-served skill gaps it repeatedly observes.
- Motivation Agent: monitors engagement-trend signals (session frequency, time-on-task decline) to detect early disengagement risk and triggers timely nudges (reminders, encouragement, difficulty adjustment) before a student disengages entirely, closing the loop back into the same event stream that drives the analytics.

Scalability & Innovation

- Scalability: keying by studentId means each learner's profile update and personalisation decision is fully independent of every other learner's, so the system scales horizontally simply by adding more Spark executors as enrolment grows into the thousands or millions.
- Innovation: using aggregateByKey (partition-local pre-aggregation) rather than groupByKey for the rolling student profile significantly reduces network shuffle volume at scale — an important, often-overlooked distributed-systems optimisation directly relevant to serving 'thousands of learners' as the question specifies.